

Q1: MobileNetV2 Transfer Learning → Plant Disease Classification

```
import tensorflow as tf

from tensorflow.keras import layers, models

from tensorflow.keras.applications import MobileNetV2

IMG_SIZE = 224

NUM_CLASSES = 5

base_model = MobileNetV2(
    weights="imagenet",
    include_top=False,
    input_shape=(IMG_SIZE, IMG_SIZE, 3)
)

base_model.trainable = False

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(NUM_CLASSES, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

Q2: DenseNet + Pixel-wise Decoder → Semantic Segmentation

```
import tensorflow as tf

from tensorflow.keras import layers, Model

from tensorflow.keras.applications import DenseNet121

IMG_SIZE = 128

NUM_CLASSES = 5

base_model = DenseNet121(
    weights="imagenet",
    include_top=False,
    input_shape=(IMG_SIZE, IMG_SIZE, 3)
)

base_model.trainable = False

inputs = layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3))

x = base_model(inputs)

x = layers.Conv2D(256, 3, padding="same", activation="relu")(x)

x = layers.UpSampling2D((2, 2))(x)

x = layers.Conv2D(128, 3, padding="same", activation="relu")(x)

x = layers.UpSampling2D((2, 2))(x)

x = layers.Conv2D(64, 3, padding="same", activation="relu")(x)

x = layers.UpSampling2D((2, 2))(x)

x = layers.Conv2D(32, 3, padding="same", activation="relu")(x)

x = layers.UpSampling2D((2, 2))(x)

outputs = layers.Conv2D(
```

```

    NUM_CLASSES,
    1,
    activation="softmax"
)(x)
model = Model(inputs, outputs)
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
model.summary()

```

Q3: Bidirectional LSTM + Encoder–Decoder → Machine Translation

```

import tensorflow as tf

from tensorflow.keras import layers, Model

VOCAB_SIZE = 10000

EMBEDDING_DIM = 128

UNITS = 256

MAX_LENGTH = 20

encoder_inputs = layers.Input(
    shape=(MAX_LENGTH,),
    name="encoder_input"
)

encoder_embedding = layers.Embedding(
    VOCAB_SIZE,

```

```

        EMBEDDING_DIM
    )(encoder_inputs)

    encoder = layers.Bidirectional(

        layers.LSTM(UNITS, return_state=True)

    )

    encoder_outputs, forward_h, forward_c, backward_h, backward_c = encoder(

        encoder_embedding

    )

    # Combine forward and backward states

    state_h = layers.Concatenate()([forward_h, backward_h])

    state_c = layers.Concatenate()([forward_c, backward_c])

    decoder_inputs = layers.Input(

        shape=(MAX_LENGTH,),

        name="decoder_input"

    )

    decoder_embedding = layers.Embedding(

        VOCAB_SIZE,

        EMBEDDING_DIM

    )(decoder_inputs)

    decoder_lstm = layers.LSTM(

        UNITS * 2,

        return_sequences=True

    )

    decoder_outputs = decoder_lstm(

        decoder_embedding,

```

```

        initial_state=[state_h, state_c]
    )
    decoder_outputs = layers.Dense(
        VOCAB_SIZE,
        activation="softmax"
    )(decoder_outputs)
    model = Model(
        [encoder_inputs, decoder_inputs],
        decoder_outputs
    )
    model.compile(
        optimizer="adam",
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    model.summary()

```

Q4: LSTM → Time-Series/Stock Prediction + BPTT

```

import numpy as np

import tensorflow as tf

from tensorflow.keras import Sequential

from tensorflow.keras.layers import LSTM, Dense, Dropout

data = np.array([
    100, 102, 101, 105, 107,
    110, 108, 112, 115, 118,

```

```

        120, 119, 123, 125, 128,
        130, 129, 133, 136, 140
    ], dtype=float)

data = data / np.max(data)

sequence_length = 5

X = []
y = []

for i in range(len(data) - sequence_length):

    X.append(data[i:i + sequence_length])

    y.append(data[i + sequence_length])

X = np.array(X)
y = np.array(y)

X = X.reshape((X.shape[0], X.shape[1], 1))

model = Sequential([

    LSTM(64, return_sequences=True,

        input_shape=(sequence_length, 1)),

    Dropout(0.2),

    LSTM(32),

    Dense(16, activation="relu"),

    Dense(1)

])

model.compile(

    optimizer="adam",

    loss="mse",

    metrics=["mae"]

```

```

)

model.summary()

model.fit(
    X,
    y,
    epochs=20,
    batch_size=4,
    verbose=1
)

prediction = model.predict(X[-1].reshape(1, sequence_length, 1))
print("Predicted normalized value:", prediction[0][0])

```

Q5: Pre-trained CNN + LSTM Encoder–Decoder → Video Captioning

```

import tensorflow as tf

from tensorflow.keras import layers, Model

from tensorflow.keras.applications import MobileNetV2

cnn = tf.keras.applications.MobileNetV2(
    weights="imagenet",
    include_top=False,
    pooling="avg"
)

cnn.trainable = False

frame_input = layers.Input(
    shape=(224, 224, 3)
)

```

```
)  
  
features = cnn(frame_input)  
  
features = layers.Dense(256, activation="relu")(features)  
  
features = layers.Reshape((1, 256))(features)  
  
encoder = layers.LSTM(  
    256,  
    return_state=True  
)  
  
encoder_output, state_h, state_c = encoder(features)  
  
VOCAB_SIZE = 5000  
  
EMBEDDING_DIM = 128  
  
MAX_LENGTH = 20  
  
caption_input = layers.Input(  
    shape=(MAX_LENGTH,) )  
  
embedding = layers.Embedding(  
    VOCAB_SIZE,  
    EMBEDDING_DIM )  
(caption_input)  
  
decoder = layers.LSTM(  
    256,  
    return_sequences=True )  
  
decoder_output = decoder(  
    embedding,
```



```
        initial_state=[state_h, state_c]
    )
    outputs = layers.Dense(
        VOCAB_SIZE,
        activation="softmax"
    )(decoder_output)
    model = Model(
        [frame_input, caption_input],
        outputs
    )
    model.compile(
        optimizer="adam",
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    model.summary()
```

